

Topic in Module 1 — Cluster and Grid Computing

Course	Modern Computing and Python Programming (26ESCC106)
Module	Module 1 — Modern Computing Approaches
Topic coverage	Cluster and Grid Computing — Introduction and Characteristics
Position in module	After Distributed Computing; before Cloud Computing
Mapped CO	CO1 — Describe modern computing approaches and their characteristics
Bloom's level	L1 — Remember, L2 — Understand
Compiled by	Mr. Manjunath Prasad H. R., Department of AI&ML, MITE
Reviewed by	1. Dr. Prashanth C. M., Principal, MITE 2. Dr. Terrence Johnson, Professor and Associate Dean — Academics 3. Dr. Ravinarayana B., Professor and Head, Dept. of CS&E

1. Introduction

The previous topic established that a distributed system is a collection of independent computers that presents itself to its users as a single coherent system [2]. That definition permits many different arrangements, and the differences between them matter. Two of these arrangements – clusters and grids – are the subject of the topic in the module. They are important in their own way, and they are also the foundation on which cloud computing, the next topic in this module, is built.

Both answer the same question: how do you obtain more computing capacity than one machine can provide? Their answers differ in one respect above all others, and it is worth stating plainly at the outset because every other difference follows from it.

A cluster is many computers owned and administered by one organisation, in one place, connected by a dedicated network. A grid is many computers owned and administered by different organisations, in different places, connected by the public network. Ownership is the distinction; everything else is a consequence of it.

2. Cluster Computing

2.1 Definition

A computer cluster is a set of complete, standalone computers connected by a dedicated high-speed local network and working together as a single integrated computing resource [1, 3]. Each machine in the cluster — called a node — has its own processor, memory, storage and operating system, and could function independently. What makes the collection a cluster is that software coordinates the nodes so that a user submits work to the cluster as a whole rather than to any particular machine.

Figure 1 illustrates the basic idea. The input data is given to the root (master) node, which divides the work and distributes the parts to the slave (worker) nodes. The nodes work on their parts in parallel and

exchange intermediate results with one another over the network where the problem requires it. The root node then collects the partial results and produces the final output.

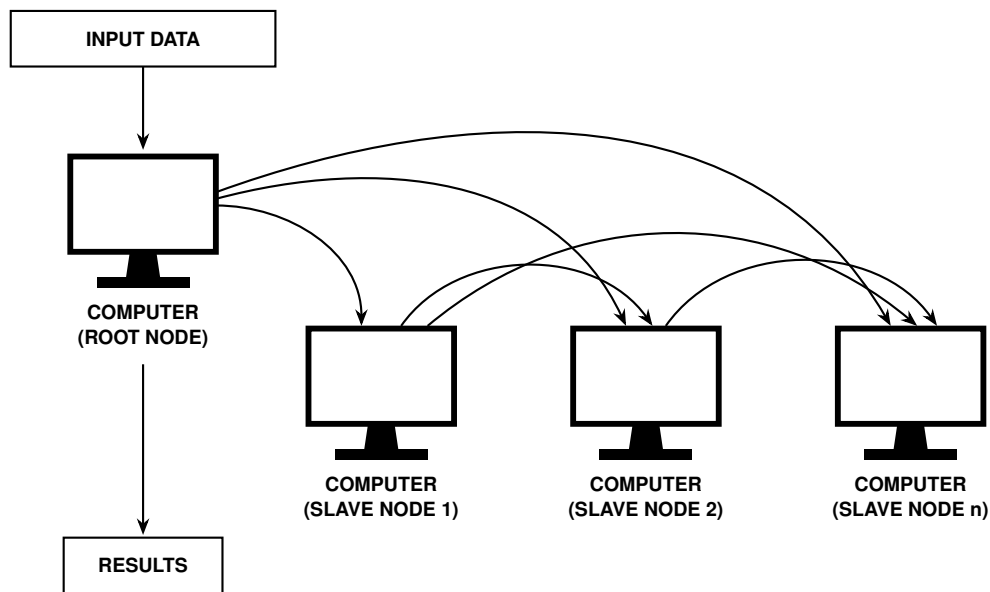


Figure 1: *Basic working model of cluster computing*

2.2 Why Clusters Exist

Until the early 1990s, an organisation needing very high computational capacity bought a supercomputer: a custom-designed machine with specially built processors and memory systems, produced in small numbers and priced accordingly. Meanwhile, mass-produced personal computer components were improving rapidly and falling in price, precisely because they were mass-produced.

The cluster is the consequence of noticing that a hundred ordinary machines cost far less than one extraordinary machine of comparable aggregate capacity. The idea was demonstrated in 1994 at NASA's Goddard Space Flight Center, where Thomas Sterling and Donald Becker built a working parallel machine from sixteen commodity personal computers and standard Ethernet running free software. The arrangement became known as a Beowulf cluster [4], and it established that serious parallel computing did not require specialised hardware.

The trade-off is real and should be understood. Commodity components are cheap but their interconnect is slower relative to processor speed than in a purpose-built supercomputer. A cluster therefore performs well on problems that divide into parts needing little communication between them, and less well on problems requiring constant exchange of intermediate results.

2.3 Components of a Cluster

Component	Function
Compute nodes	Standalone computers that perform the actual work; each has its own processor, memory and operating system
Head or master node	The single point of entry through which users log in and submit work; it does not usually perform computation itself
High-speed interconnect	A dedicated private network joining the nodes — typically 10-, 25- or 100-Gigabit Ethernet, or InfiniBand where very low latency is required
Cluster middleware	Software that makes the collection behave as one machine, providing the single system image and handling node failure
Resource manager and job scheduler	Accepts submitted jobs, queues them, allocates nodes and manages execution — for example Slurm or PBS
Parallel programming environment	Libraries that let one program run across many nodes, most commonly MPI for communication between nodes and OpenMP within a node
Shared storage	A file system visible to every node, so that a job can read its input and write its output regardless of where it runs

2.4 Cluster Architecture

Figure 2 shows the generic architecture of a cluster, as presented by Yeo and Buyya *et al.* [3]. It is read from the bottom up. The nodes — PCs or workstations — are joined by the cluster interconnection network or switch. Each node has network interface hardware that attaches it to this network, communication software that provides fast and reliable data exchange between nodes, and its own operating system. Above the individual nodes sits the cluster middleware, which provides the single system image and the availability infrastructure: it makes the separate machines appear as one computer and keeps the service running when a node fails. On top of the middleware, sequential applications run directly, while parallel applications run through a parallel programming environment — compilers, message-passing libraries such as MPI, debuggers and performance tools — that lets one program use many nodes at once.

Figure 3 shows how such a cluster is typically deployed in practice. Users never connect to the compute nodes directly; they log in to the head node over the campus network or the Internet and submit their jobs to the scheduler running there. The head node, the shared storage and every compute node are joined by the private high-speed interconnect, which carries the messages exchanged between the parts of a running job as well as the traffic between each node and the shared file system. The head node and the shared storage are where much of the middleware and scheduling software of Figure 2 actually runs. Everything inside the dashed boundary belongs to one organisation and sits in one facility.

2.5 Single System Image

The property that distinguishes a cluster from a merely networked room of computers is the single system image: the cluster presents itself to the user as one machine. The user does not choose which nodes will

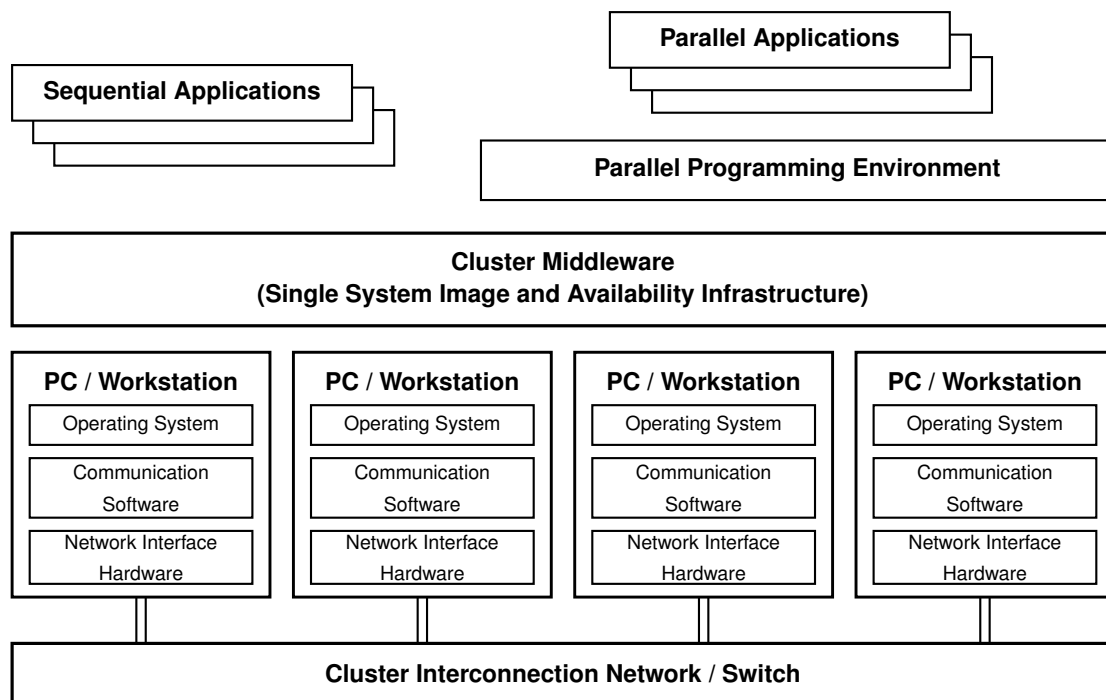


Figure 2: Cluster computer architecture (redrawn from [3]; the figure originates in [7])

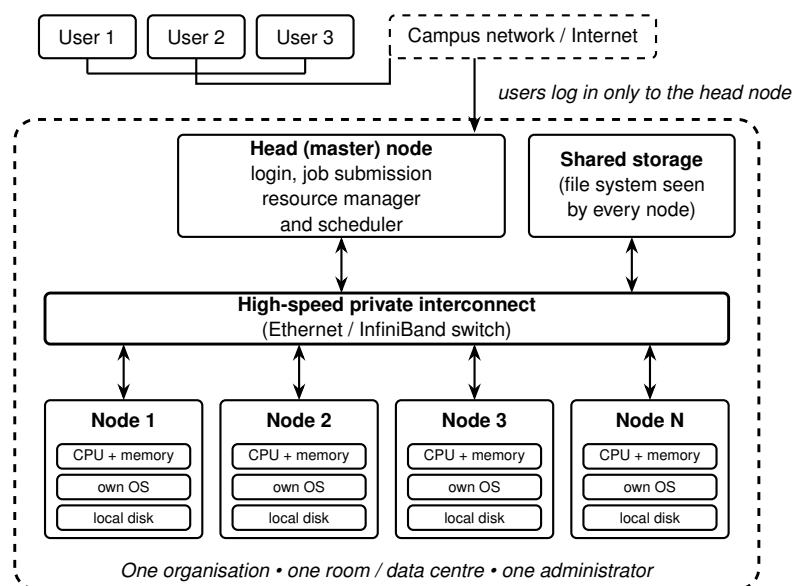


Figure 3: Physical organisation of a cluster

run their program, does not log in to individual machines, and does not manually distribute data. They describe the resources their job needs and submit it; the scheduler decides where it runs. This abstraction is what makes a cluster usable, and providing it is the main job of the cluster middleware [3, 1].

2.6 Characteristics of Cluster Computing

- **Single administrative domain** — all nodes are owned, funded and administered by one organisation, under one security policy
- **Physical co-location** — nodes sit in one room or one data centre, typically in the same rack or set of

racks

- **Homogeneity** — nodes are usually identical or near-identical in hardware, operating system and configuration, which simplifies scheduling and programming
- **Tight coupling** — a dedicated, private, high-bandwidth and low-latency interconnect, not shared with general network traffic
- **Dedicated resources** — nodes exist to serve the cluster and are not used for anything else
- **Centralised scheduling** — one resource manager has complete knowledge of and authority over every node
- **Single system image** — the collection appears to the user as one computer
- **High availability** — the failure of one node need not stop the cluster, because work can be restarted elsewhere
- **Incremental scalability** — capacity grows by adding nodes, within the limits of space, power and interconnect

2.7 Types of Clusters

Clusters are built for three broad purposes [1].

Type	Purpose	Typical use
High-performance (HPC) cluster	Maximise computational throughput on a single large problem by dividing it across nodes	Scientific simulation, weather modelling, molecular dynamics, model training
Load-balancing cluster	Distribute many independent incoming requests across nodes so that no node is overwhelmed	Web servers, application servers, online services
High-availability (failover) cluster	Maintain service continuity by having standby nodes take over when an active node fails	Database servers, banking and other systems that must not stop

2.8 Examples of Clusters

- Beowulf clusters — the original 1994 NASA machine and the many university and departmental clusters modelled on it, built from commodity PCs running Linux
- The PARAM series developed by C-DAC under the National Supercomputing Mission, for example PARAM Siddhi-AI at C-DAC Pune and PARAM Pravega at IISc Bengaluru — high-performance clusters of many compute nodes joined by a high-speed interconnect [11]
- Most of the world's fastest computers listed in the TOP500 ranking, which are built as clusters rather than as single custom machines [10]

- Web server farms behind a load balancer — search engines, e-commerce and railway or airline booking sites spread incoming requests across many identical servers (load-balancing clusters)
- High-availability database clusters, such as Oracle Real Application Clusters, used for core banking and reservation systems where a standby node must take over if an active node fails

2.9 Advantages and Limitations

Advantages

- Substantially better price-to-performance ratio than a purpose-built supercomputer
- Capacity can be increased in small increments rather than by replacing the whole machine
- Redundancy is inherent, so availability is high if the software is configured for it
- Built from standard components with standard software, so expertise and spare parts are widely available

Limitations

- Bounded by one organisation's budget, floor space, power supply and cooling capacity
- Programs must be written explicitly for parallel execution; existing sequential software does not benefit
- The interconnect eventually limits how far a tightly coupled problem can be scaled
- Capacity is fixed once purchased — it must be sized for peak demand and is idle at other times

3. Grid Computing

3.1 Definition

Grid computing is the coordinated sharing of computing resources that belong to different organisations, in different locations, in order to solve a common problem. The resources are federated rather than owned in common: each participating institution keeps ownership of and authority over its own machines, and contributes their use under agreed terms.

The term was introduced in the late 1990s, principally through the work of Ian Foster and Carl Kesselman [5], who characterised the field by the problem of coordinated resource sharing across institutional boundaries [6].

3.2 The Power Grid Analogy

The name is borrowed from the electrical power grid, and the analogy explains the ambition [5]. When you switch on a light, you do not know or care which generating station supplied the electricity, whether it came from one source or several, or where those sources are. You draw power from a socket, it is metered, and you are billed. The generation, transmission and coordination are invisible.

Grid computing proposed the same relationship with computation: a researcher would draw processing power from a socket without knowing which institution's machines were serving them. It is worth noting

that this ambition was only partially realised. Electricity is a uniform commodity and computing is not — machines differ in architecture, software and policy in ways that generators do not — and the seamlessness the analogy promised proved difficult to deliver. The vision was largely fulfilled later, by cloud computing, and by a different route.

3.3 The Virtual Organisation

The central concept in grid computing is the virtual organisation: a set of individuals or institutions, drawn from different real organisations, who agree to share specified resources under specified rules for a specified purpose. A virtual organisation is defined by its sharing agreement, not by a legal entity or a physical site [6].

A concrete example makes this clear. Physicists at twenty universities in twelve countries collaborate on one experiment. Each university owns a cluster. No university is willing to hand over control of its cluster to any other, and each has its own security policy, its own users and its own local commitments. Yet the experiment needs more capacity than any one of them has. The virtual organisation is the arrangement under which those twenty clusters serve the experiment while each remains under its owner's control. Grid middleware exists to make that arrangement work in practice.

3.4 Grid Architecture

A grid can be described from two viewpoints: how the participating sites are arranged, and how the grid software is organised. Figure 4 shows the first. Each site — a university cluster, a research laboratory's supercomputer, a data archive, a scientific instrument, or a pool of contributed desktop PCs — remains under its owner's control and keeps its own local scheduler and policy. A layer of grid middleware at each site acts as a gateway, translating requests from the grid into local actions that the owner has agreed to permit. The sites are linked only by the wide-area network, and together they form the virtual organisation described above.

A user does not approach the sites one by one. They authenticate once, using a digital certificate, and submit work to a grid portal or resource broker. The broker asks the information service which resources are currently available and suitable, and the security service establishes that the user is trusted at each site chosen, so that one login is accepted everywhere (single sign-on). The broker then dispatches the job, and the necessary data, across the network to those sites.

3.5 Characteristics of Grid Computing

- **Multiple administrative domains** — no single owner or authority; each site retains autonomy over its own resources and enforces its own policies
- **Geographical distribution** — resources are spread across cities, countries or continents
- **Heterogeneity** — nodes differ in hardware architecture, operating system, software stack and configuration
- **Loose coupling** — sites are connected by wide-area networks or the public Internet, with latency orders of magnitude higher than a cluster interconnect

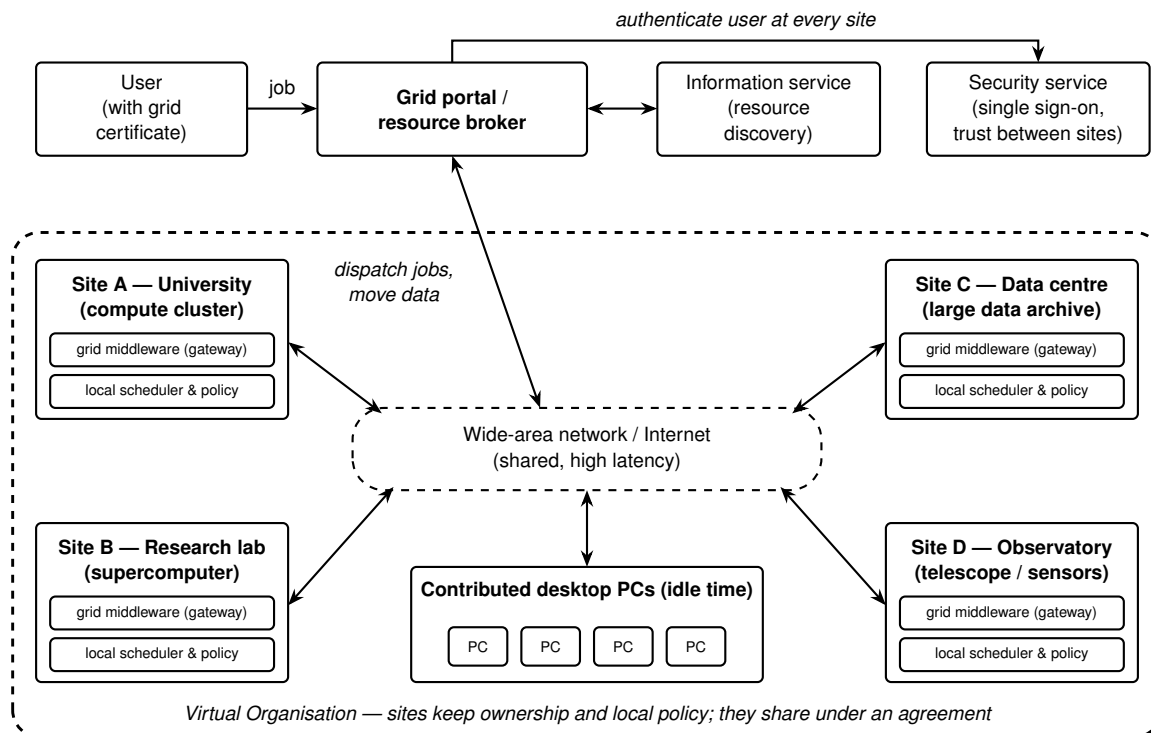


Figure 4: Architecture of a grid — independent sites joined into a virtual organisation through grid middleware

- **Dynamic and unpredictable membership** — resources join, leave and fail without notice, so the grid must tolerate their disappearance
- **Non-dedicated resources** — contributed machines usually have local work of their own, and grid jobs run on spare capacity at lower priority
- **Policy-based sharing** — what may be used, by whom, and for what is governed by negotiated agreements rather than by ownership
- **Decentralised scheduling** — no single scheduler has authority over all resources; allocation is negotiated between sites
- **Reliance on standard protocols and middleware** — the Globus Toolkit and the Open Grid Services Architecture were the principal efforts to standardise this
- **Best-effort service** — there is generally no guarantee of when a job will run or on what, because no participant has undertaken to provide capacity on demand

3.6 Types of Grids

Grids are classified by what they share [1, 5].

Type	Description
Computational grid	Aggregates processing capacity across sites to execute computationally intensive work
Data grid	Provides controlled, coordinated access to very large datasets held at multiple institutions
Collaboration or service grid	Shares instruments, services and applications — telescopes, sensors, specialised software — rather than raw computation
Desktop or volunteer grid	Uses idle time on ordinary personal computers contributed by the public, coordinated over the Internet

3.7 Examples

- The Worldwide LHC Computing Grid, which distributes and processes data from the Large Hadron Collider across roughly 170 computing centres in more than 40 countries [8]
- The European Grid Infrastructure, federating computing resources across European research institutions [9]
- GARUDA, the Indian national grid initiative coordinated by C-DAC, connecting research and academic institutions across the country [11]
- Volunteer computing projects for protein folding and radio-signal analysis, run on donated capacity from personal computers

3.8 Advantages and Limitations

Advantages

- Exploits existing resources that would otherwise sit idle, at little additional hardware cost
- Aggregate capacity far exceeding what any single participant could afford to build
- Enables collaboration across institutional and national boundaries without transferring ownership
- No participant is required to purchase the whole of the capacity it occasionally needs

Limitations

- Heterogeneity makes software difficult to write, deploy and validate across sites
- Security and trust are hard: a user authenticated at one institution must be granted appropriate rights at another that does not directly know them
- Wide-area latency makes the grid unsuitable for tightly coupled problems requiring frequent communication
- Middleware is complex and demands specialist administration at every participating site
- No guarantee of availability or performance, because resources are contributed rather than committed

- Accounting is difficult — measuring and settling who consumed whose capacity never worked cleanly, which limited participation beyond publicly funded research

4. Cluster Computing versus Grid Computing

The table below draws together the differences established in the two previous sections [1, 6].

Aspect	Cluster computing	Grid computing
Ownership	One organisation	Many independent organisations
Administration	Single administrative domain	Multiple autonomous domains
Location	Co-located in one facility	Geographically distributed
Hardware and software	Homogeneous	Heterogeneous
Interconnect	Dedicated high-speed LAN, low latency	Wide-area network or Internet, high latency
Coupling	Tight	Loose
Resource dedication	Dedicated to the cluster	Shared with the owner's own local workload
Scheduling	Centralised, with full authority	Decentralised and negotiated
Membership	Static and known	Dynamic and variable
Security	One policy, one trust domain	Cross-domain trust and identity federation required
Service guarantee	Predictable and controlled	Best effort
User view	A single machine	A pool of resources reached through middleware
Suitable workload	Tightly coupled parallel problems	Independent, coarse-grained tasks

5. Review Questions

CO1 — Describe modern computing approaches and their characteristics

Bloom's Level L1 — Remember; L2 — Understand

1. Define cluster computing and explain what distinguishes a cluster from a set of networked computers in the same room.
2. What is a Beowulf cluster? Explain the economic argument that motivated it.
3. List the principal components of a cluster and state the function of each.
4. Explain the term single system image and describe why it matters to a cluster user.
5. List any six characteristics of cluster computing.

6. Differentiate between high-performance, load-balancing and high-availability clusters, giving one application of each.
7. State any four limitations of cluster computing.
8. Define grid computing and explain the origin of the term.
9. Explain the power grid analogy. Identify one respect in which the analogy does not hold.
10. What is a virtual organisation? Explain with an example why it is the central concept in grid computing.
11. List any six characteristics of grid computing.
12. Differentiate between a computational grid and a data grid.
13. What is volunteer computing? Explain what kind of problem it is suited to and why.
14. Compare cluster and grid computing with respect to ownership, coupling, homogeneity, scheduling and service guarantee.
15. Explain why security is substantially harder to provide in a grid than in a cluster.
16. Why is grid computing unsuitable for tightly coupled parallel problems?
17. Give one real-world example each of a high-performance, a load-balancing and a high-availability cluster.
18. With a neat diagram, explain the architecture of a computer cluster, showing both its physical organisation and its software layers.
19. With a neat diagram, explain how a user's job is executed on a grid, stating the role of the resource broker, the information service, the security service and the grid middleware at each site.
20. Explain the layered grid architecture with a diagram, stating the function of each layer.

6. References and Further Reading

Items 1 to 7 are the sources used in preparing these notes and are suggested for further reading; section and chapter numbers are indicative. Items 8 to 11 are the project and ranking sites from which the examples and figures quoted in the text were taken.

- [1] K. Hwang, G. C. Fox and J. J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, 2012 — Chapter 2 on computer clusters for scalable parallel computing, and Chapter 7 on grid computing systems and resource management.
- [2] R. Buyya, C. Vecchiola, S. T. Selvi, S. Poojara and S. N. Srirama, *Mastering Cloud Computing*, 2nd Edition, Morgan Kaufmann, 2026 — Chapter 2 on the principles of parallel and distributed computing.
- [3] C. S. Yeo, R. Buyya, H. Pourreza, R. Eskicioglu, P. Graham and F. Sommers, "Cluster Computing: High-Performance, High-Availability, and High-Throughput Processing on a Network of Computers", in A. Y. Zomaya (ed.), *Handbook of Nature-Inspired and Innovative Computing*, Springer, 2006 — the source of Figure 2.

- [4] T. Sterling, D. J. Becker, D. Savarese, J. E. Dorband, U. A. Ranawake and C. V. Packer, “BE-OWULF: A Parallel Workstation for Scientific Computation”, *Proceedings of the 24th International Conference on Parallel Processing*, 1995.
- [5] I. Foster and C. Kesselman (eds.), *The Grid: Blueprint for a New Computing Infrastructure*, 2nd Edition, Morgan Kaufmann, 2004.
- [6] I. Foster, C. Kesselman and S. Tuecke, “The Anatomy of the Grid: Enabling Scalable Virtual Organizations”, *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001 — the source of the layered architecture in Figure ??.
- [7] R. Buyya (ed.), *High Performance Cluster Computing: Architectures and Systems*, Volume 1, Prentice Hall PTR, 1999 — where the architecture of Figure 2 first appeared.
- [8] Worldwide LHC Computing Grid, CERN. <https://wlcg.web.cern.ch/>
- [9] EGI Federation (European Grid Infrastructure). <https://www.egi.eu/>
- [10] TOP500 list of the world’s most powerful computer systems. <https://www.top500.org/>
- [11] Centre for Development of Advanced Computing (C-DAC) — PARAM systems under the National Supercomputing Mission, and the GARUDA national grid initiative. <https://www.cdac.in/>